In [47]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

In [55]:
```
# Prepare the data
X_simple = df[['rm']]  # Independent variable
y = df['medv']         # Target variable

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_simple, y, test_siz

# Initialize and train
simple_model = LinearRegression()
simple_model.fit(X_train, y_train)

# Predict
y_pred_simple = simple_model.predict(X_test)
```


```
Simple Regression R2 Score: 0.5960
```

In [ ]:
```
# Plotting the results
plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='gray', label='Actual Data')
plt.plot(X_test, y_pred_simple, color='red', linewidth=2, label='Regressi
plt.xlabel('Average Number of Rooms (RM)')
plt.ylabel('House Price (MEDV)')
plt.title('Simple Linear Regression: RM vs Price')
plt.legend()
plt.show()

print(f"Simple Regression R2 Score: {r2_score(y_test, y_pred_simple):.4f}
```

In [59]:
```
# Prepare the data (using all features)
X_multi = df.drop('medv', axis=1)
y = df['medv']

# Split data
X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(X_multi, y, t

# Initialize and train
multi_model = LinearRegression()
multi_model.fit(X_train_m, y_train_m)

# Predict
y_pred_multi = multi_model.predict(X_test_m)

# Evaluation Metrics
mse = mean_squared_error(y_test_m, y_pred_multi)
r2 = r2_score(y_test_m, y_pred_multi)

print(f"Multiple Regression Mean Squared Error: {mse:.4f}")
print(f"Multiple Regression R2 Score: {r2:.4f}")
```

```
Multiple Regression Mean Squared Error: 23.4117
Multiple Regression R2 Score: 0.7399
```

In [61]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Predicted vs Actual
ax1.scatter(y_test_m, y_pred_multi, alpha=0.7, color='teal')
ax1.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2) # Identity
ax1.set_xlabel('Actual Prices')
ax1.set_ylabel('Predicted Prices')
ax1.set_title('Multiple Regression: Predicted vs Actual')

# Plot 2: Residual Plot
residuals = y_test_m - y_pred_multi
ax2.scatter(y_pred_multi, residuals, alpha=0.7, color='purple')
ax2.axhline(y=0, color='black', linestyle='--')
ax2.set_xlabel('Predicted Prices')
ax2.set_ylabel('Residuals (Errors)')
ax2.set_title('Residual Analysis')

plt.tight_layout()
plt.show()
```